

系统延时

系统延时

一、学习目标

滴答定时器原理

延时的作用

二、硬件搭建

三、实验步骤

1. 打开SYSCONFIG配置工具

2. 滴答定时器配置

3. 写入程序

4. 编译

四、程序分析

五、实验现象

一、学习目标

1.学习MSPM0G3507主板的引脚基本使用。

2.了解滴答定时器的用法。

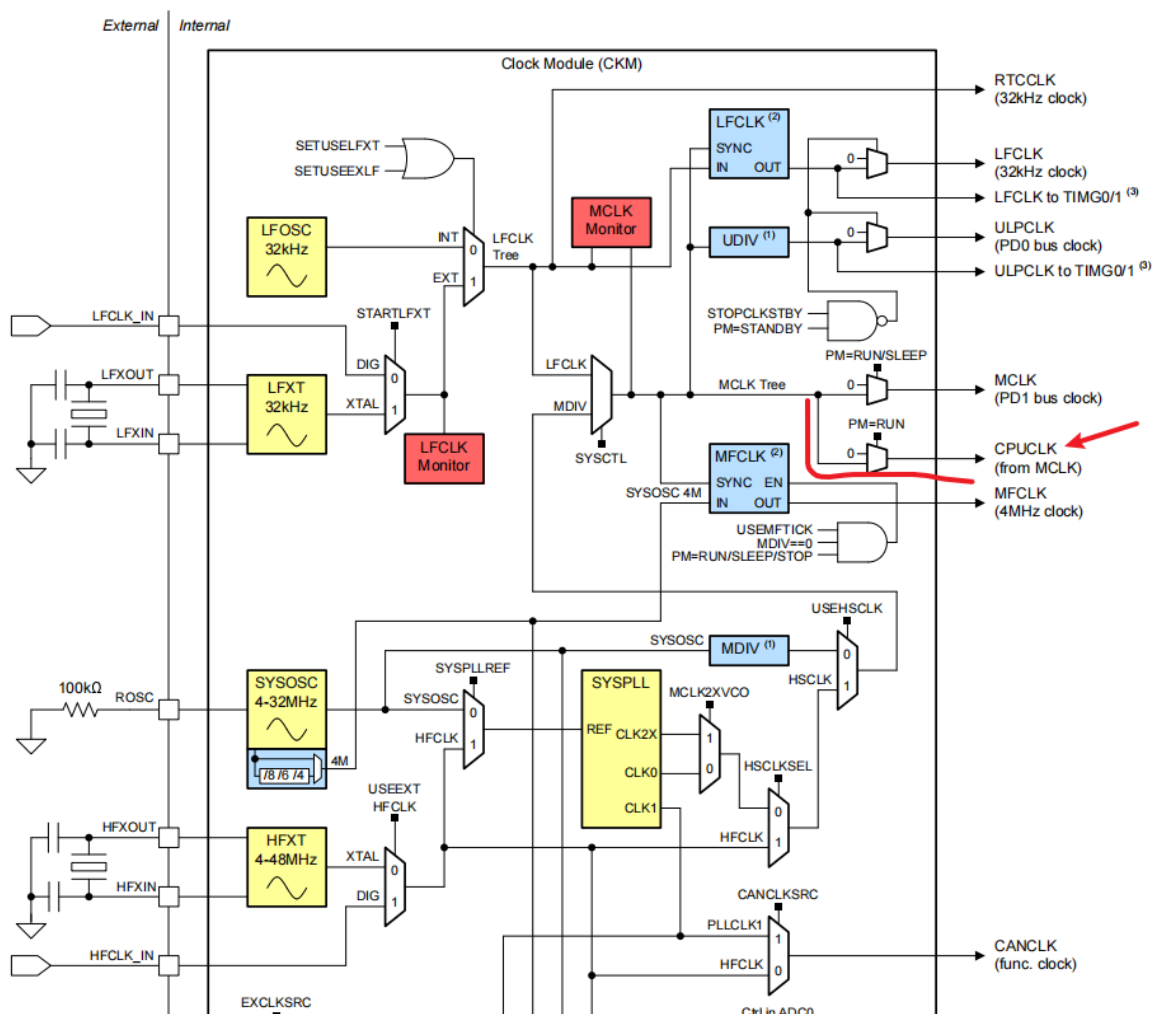
滴答定时器原理

滴答定时器是一种常用的硬件计时器，用于生成固定周期的中断或触发定时操作。它的基本工作原理是通过主时钟（MCLK）计数，以实现精确的时间间隔。

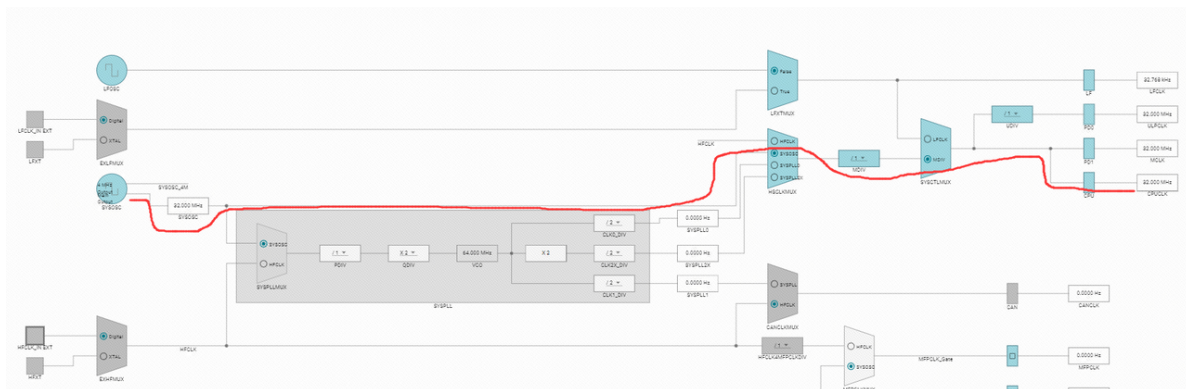
SysTick 定时器是 Cortex-M0 处理器内置的标准定时器，作为一个 24 位向下计数器，它具备自动重载功能，并能够屏蔽系统中断。所有基于 M0 内核的控制器都配备有 SysTick 定时器，这使得在不同设备间移植程序变得更加简单。SysTick 定时器通常用于操作系统，为任务调度提供必要的时钟节拍，起到“心跳”的作用，保证系统按时执行任务。由于所有 M0 内核的芯片都包含 SysTick 定时器，因此在移植时，程序不需要重新配置定时器，相比普通定时器，移植难度较低。

在系统时钟的配置中，RCU 通过 MCLK 提供 Cortex 系统定时器（SysTick）的外部时钟，默认情况下 MCLK 的频率为 32MHz。通过设置 SysTick 控制和状态寄存器，可以控制或读取定时器状态。关于系统时钟的详细信息，可以参考用户手册中的时钟树部分。

时钟树：



工程中默认的时钟路径:



SysTick 定时器的工作方式为：在设定初值并使能后，每经过一个系统时钟周期，计数值减 1。当计数器达到 0 时，SysTick 会自动重新装载初值并继续计数，同时将内部的 COUNTFLAG 标志位置为 1，触发中断（前提是中断已启用）。

延时的作用

延时函数是一种用在程序中引入人为延时的函数，其主要的作用就是让程序在指定时间段内暂停执行。简单来说，延时函数的作用就在于等，实际上就是要等一段时间再执行接下来的代码。

- **硬件响应等待：**许多硬件设备需要一定时间来完成操作或稳定。例如，向一个显示屏发送初始化命令后，如果立刻开始发送显示数据，可能会出现显示异常。通过延时可以让硬件有充足时间完成初始化，确保正常工作。
- **定时任务实现：**延时可以帮助实现固定时间间隔的操作。例如，一个微控制器控制的自动门，每隔 5 秒检查红外传感器是否检测到障碍物。如果未检测到，门保持关闭状态；如果检测到，门则开启并保持 2 秒，再自动关闭。

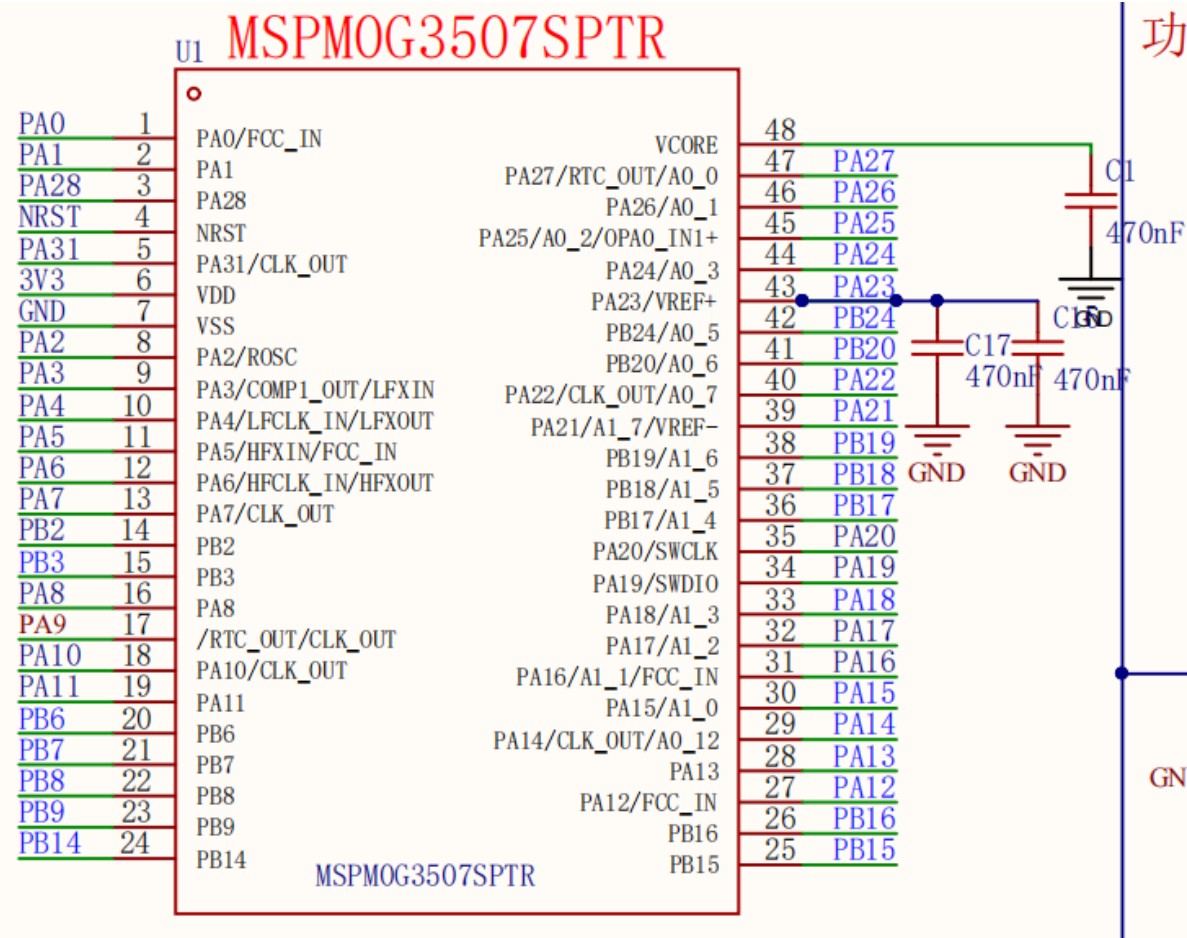
- 用户交互优化：在用户交互场景中，延时能够实现更好的体验。例如，当一个按钮被按下时，通过延时实现“防抖”功能，避免由于机械振动产生的多次触发。此外，在状态指示灯切换中，延时用于控制亮灭频率，使灯光闪烁更平滑且易于观察。

二、硬件搭建

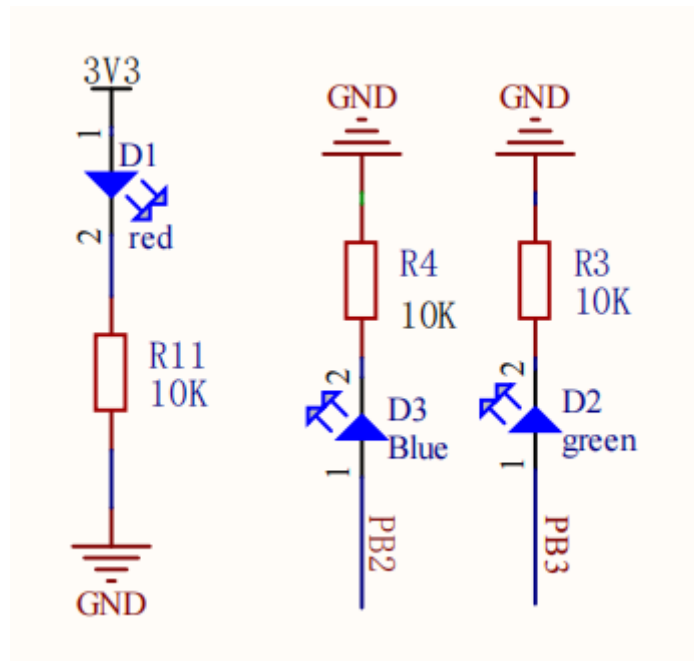
本次课程无需额外的硬件设备，直接利用MSPM0G3507主板上的板载LED灯即可，上一个课程我们用的是不精确延时，本节课程搭配滴答定时器实现精确延时。

我们在主板上设置了两个LED用户灯，用户可以自行DIY其功能。下面以**LED1**例。

MSPM0G3507主控图



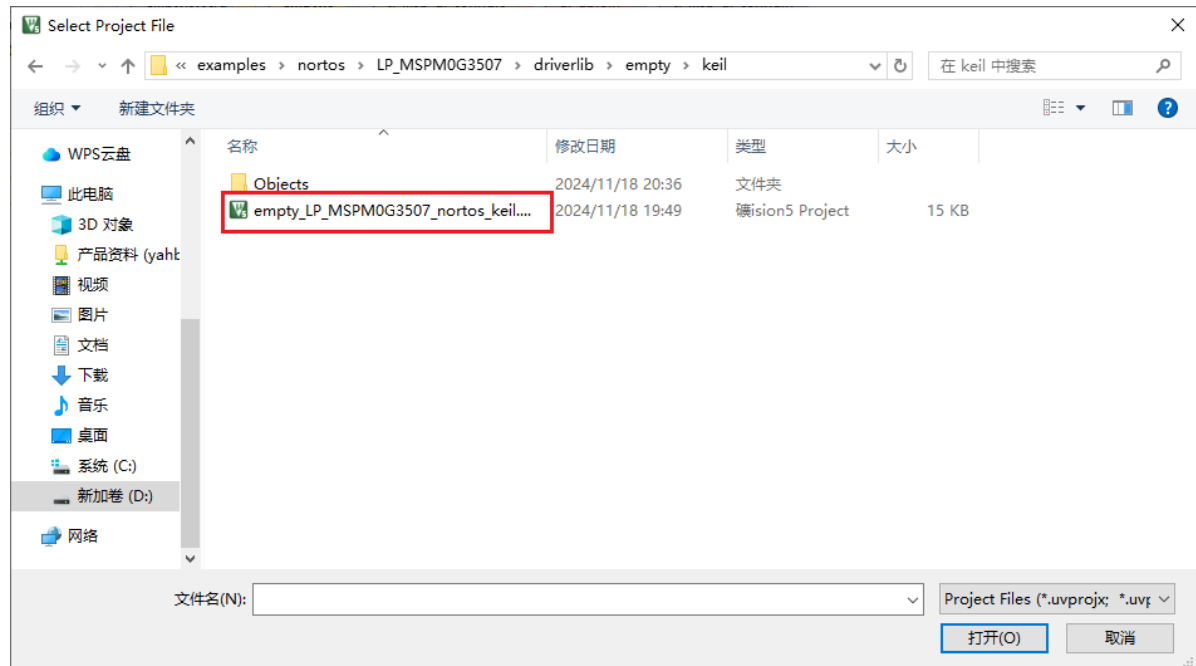
LED部分原理图



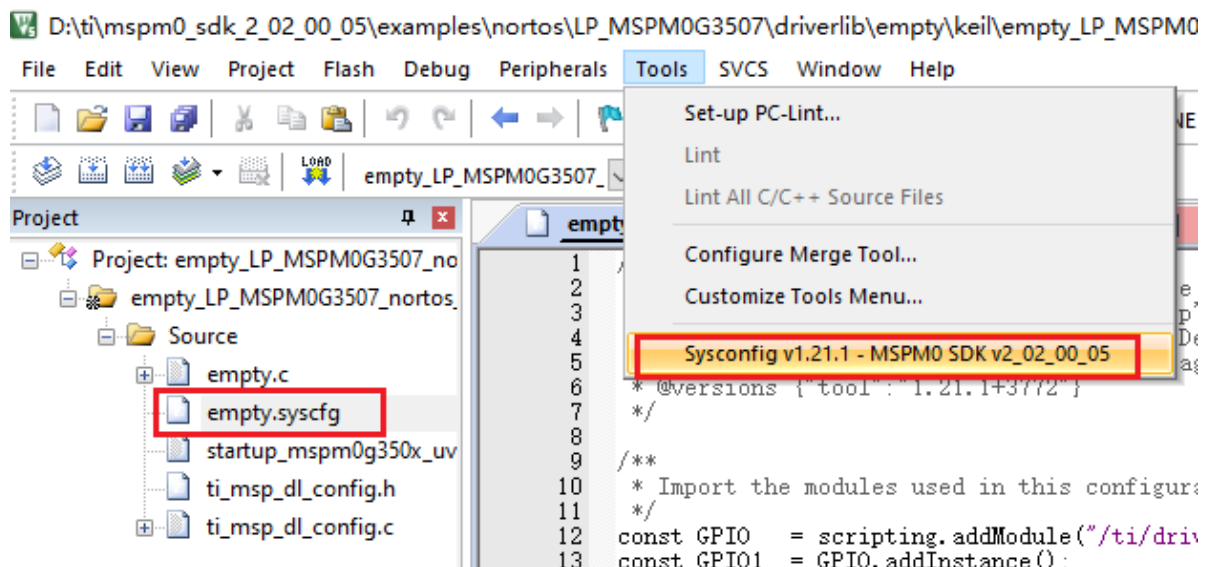
三、实验步骤

1. 打开SYSCONFIG配置工具

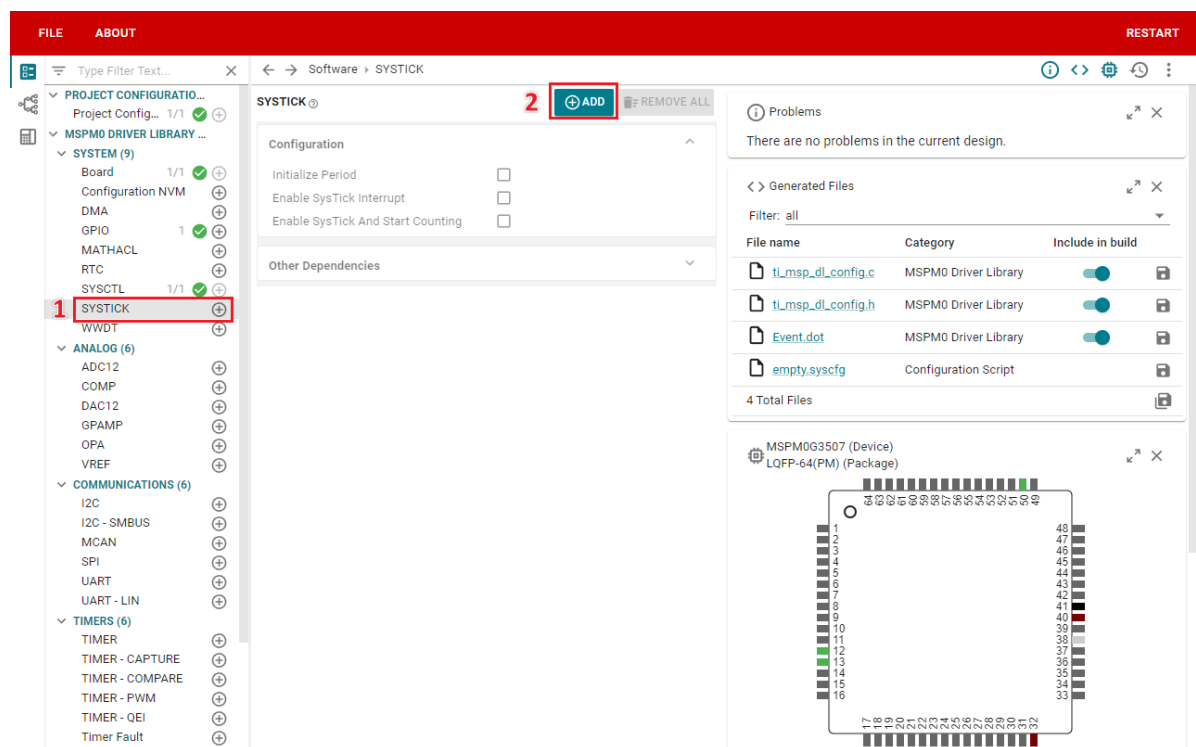
在KEIL中打开SDK中的空白工程 empty。



选中之后打开，在keil界面打开 empty.syscfg 文件，在 **empty.syscfg** 文件打开情况下，再在 Tools 栏选择打开 SYSCONFIG 的GUI界面。

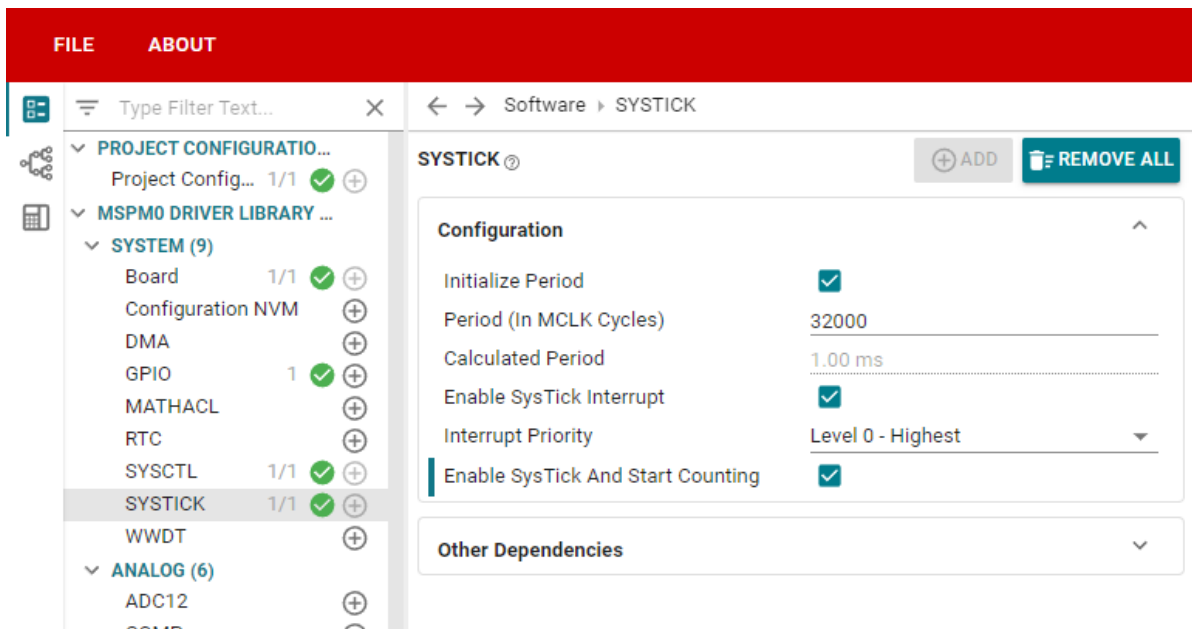


在 SYSCONFIG 中，左侧选择 MCU 外设，找到 SYSTICK 选项并点击进入。在 SYSTICK 中点击 ADD，添加滴答定时器。



2. 滴答定时器配置

滴答定时器是使用MCLK计时，而MCLK为32MHz，即每计一次数花费的时间为 $1 \div 32,000,000 = 0.00000003125$ S. 假设我们设置要计数的值为32000，则计32000个数会花费的时间为： $32000 \times 0.00000003125 = 0.001$ S = 1MS. 所以如果我们要设置1ms的滴答定时器就设置滴答定时器的计数值为32000。



参数说明：

Initialize Period: 初始定时器周期。

Period (In MCLK Cycles): 设置周期计数值，这里我们设置为32000。

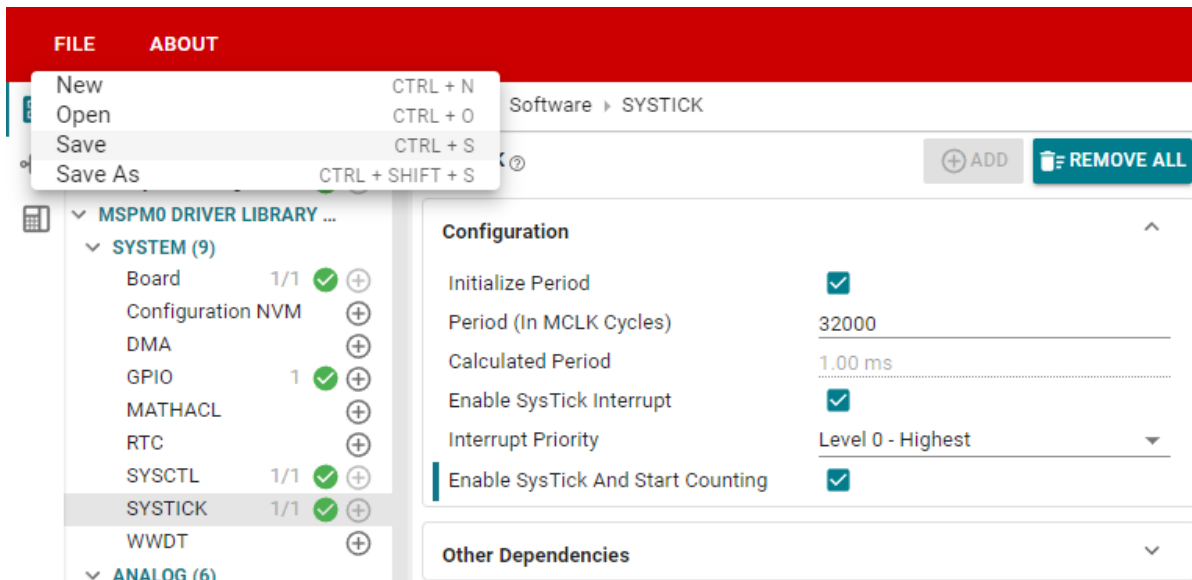
Calculated Period: 计数花费的时间。软件会自动计算出该值。

Enable SysTick Interrupt: 是否开启滴答定时器中断。

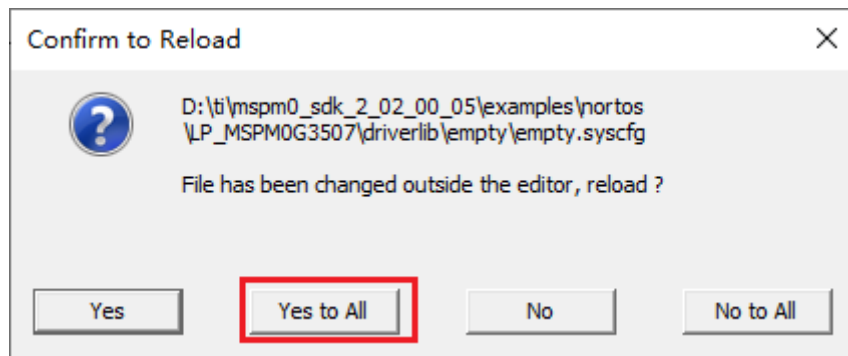
Interrupt Priority: 选择中断优先级。这里选择了最高优先级的中断。

Enable SysTick And start Counting: 是否开启滴答定时器并开始计数。

点击 SAVE 将 SYSCONFIG 中的配置保存，之后关掉 SYSCONFIG 回到keil。



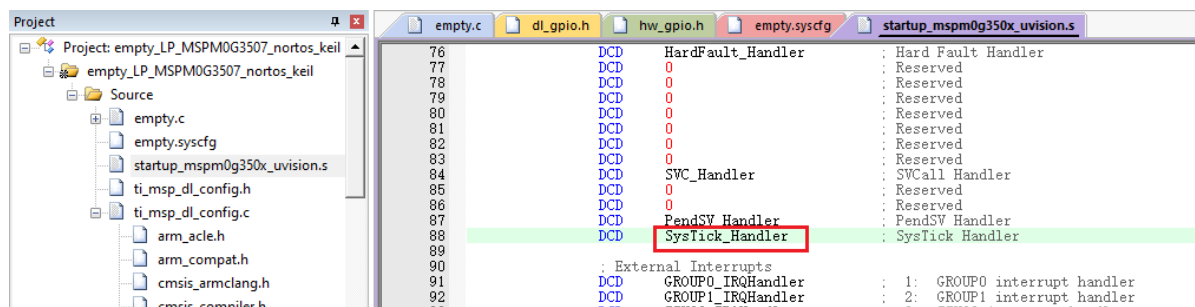
在弹窗中选择 Yes to All



同样我们还需要确认ti_msp_dl_config.c和ti_msp_dl_config.h文件是否更新。直接编译，编译就会自动更新到keil中。如果没有更新，我们也可以把 SYSCONFIG 中的文件内容复制过来。

3. 写入程序

配置好滴答定时器后，我们还需要手动编写滴答定时器的中断服务函数。因为我们启用了滴答定时器的中断，当滴答定时器的计数值从设置的初值减至 0 时，就会触发一次中断，触发后会自动执行中断服务函数。每个中断对应的中断服务函数名称通常是固定的，不能随意修改，否则将无法正确进入中断服务函数。中断服务函数的具体名称可以在工程的 `startup_mspm0g350x_uvision.s` 文件中找到，该文件中列出了各个中断源对应的中断服务函数名称。



在empty.c文件中，写入以下代码

```
#include "ti_msp_dl_config.h"

volatile unsigned int delay_times = 0;

//搭配滴答定时器实现的精确ms延时 Accurate ms delay with tick timer
void delay_ms(unsigned int ms)
{
    delay_times = ms;
    while( delay_times != 0 );
}

int main(void)
{
    SYSCFG_DL_init();

    while (1)
    {
        //LED引脚输出高电平 LED pin outputs high level
        DL_GPIO_setPins(LED1_PORT, LED1_PIN_2_PIN);
        delay_ms(1000);
        //LED引脚输出低电平 LED pin outputs low level
        DL_GPIO_clearPins(LED1_PORT, LED1_PIN_2_PIN);
        delay_ms(1000);
    }
}
```

```

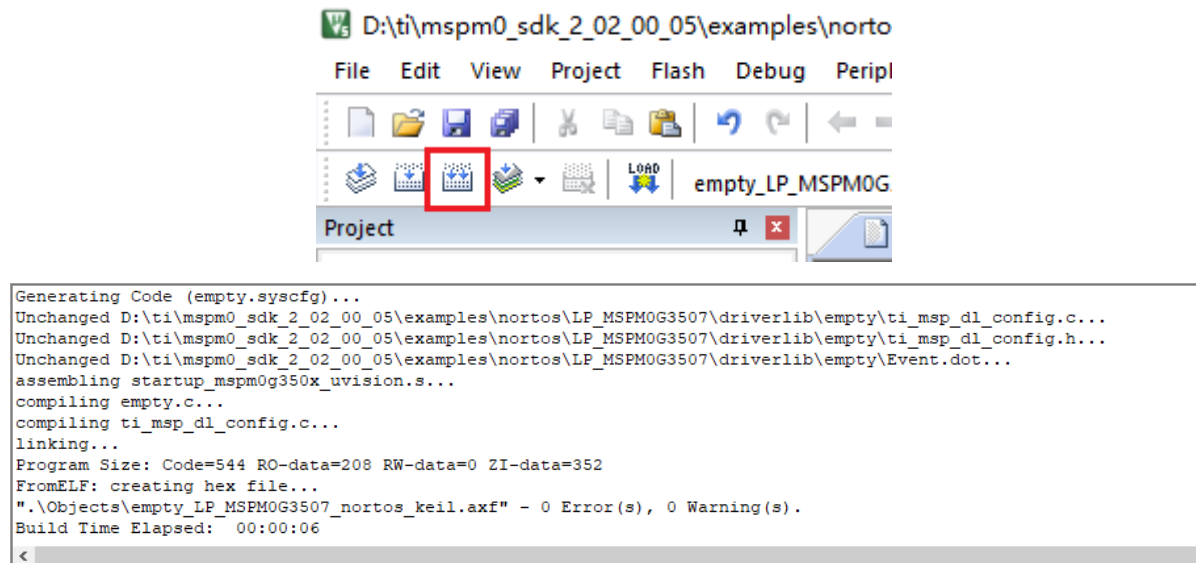
}

void SysTick_Handler(void)
{
    if( delay_times != 0 )
    {
        delay_times--;
    }
}

```

4. 编译

点击 Rebuild 图标。出现以下提示表示编译完成，并且没有错误。



四、程序分析

- empty.c

```

volatile unsigned int delay_times = 0;

//搭配滴答定时器实现的精确ms延时 Accurate ms delay with tick timer
void delay_ms(unsigned int ms)
{
    delay_times = ms;
    while( delay_times != 0 );
}

int main(void)
{
    SYSCFG_DL_init();

    while (1)
    {
        //LED引脚输出高电平 LED pin outputs high level
        DL_GPIO_setPins(LED1_PORT, LED1_PIN_2_PIN);
        delay_ms(1000);
        //LED引脚输出低电平 LED pin outputs low level
        DL_GPIO_clearPins(LED1_PORT, LED1_PIN_2_PIN);
        delay_ms(1000);
    }
}

```



```

    }
}

void SysTick_Handler(void)
{
    if( delay_times != 0 )
    {
        delay_times--;
    }
}

```

定义一个全局变量 `delay_times`，用来记录延时剩余的毫秒数。

`delay_ms()`：实现了精确的毫秒级延时。

`main()`：初始化系统之后调用 `DL_GPIO_setPins` 指定引脚输出高低电平来控制LED灯的亮灭。

`SysTick_Handler()`：在中断中对 `delay_times` 递减，如果 `delay_times` 为非零，每次中断将其减1，当减为0是，延时完成，`delay_ms()` 中的 `while` 循环退出。

五、实验现象

程序下载完成之后，可以看到MSPM0开发板上的LED1灯1s亮1s灭，反复循环。